

Event Management Backend

Ticketing and Reservations — Project Brief for Interns

Objective: Build a reliable backend service that enables organizations to sell event tickets with a smooth reservation and purchase experience, prevents oversell under concurrency, and gives the business the controls it needs to run events safely — packaged and runnable in Docker.

1. Key Stakeholders and Their Needs

Stakeholder	Needs	Success Criteria
Event organizer	Accurate capacity control and simple ticket configuration; refunds work correctly.	No oversells; correct inventory after refunds.
End customer (attendee)	Fast reservation flow, reliable confirmation, clear hold windows.	Low checkout failures; predictable hold windows.
Operations / Support	Audit logs for every inventory change so issues can be traced.	Every inventory change is traceable.
Finance	Idempotent order records — no duplicate orders from retried requests.	No duplicate orders from retried requests.
Security & Compliance	No raw payment data stored; simulated payment capture only.	No sensitive payment data persisted.

2. Business Rules and Policies

Ticket Capacity

- Total capacity is set per ticket type and is **immutable** for reporting.
- Available inventory is the business-facing number that decreases only on **confirmed purchases**.

Reservation Hold Policy

- Hold window default: **10 minutes** (configurable per event).
- Holds do not permanently reduce available inventory. Holds expire automatically and are cleaned up by a background job.

Purchase Confirmation

- Purchases must be **idempotent**. Clients must supply an **Idempotency-Key**. Duplicate requests with the same key return the same order result.
- Payment capture is **simulated** for this project; design must show where a real payment provider would be integrated.

Refunds and Cancellations

- Full-order refunds are recorded as separate transactions and increase available inventory **atomically**.

Overbooking Policy

- **Zero tolerance** for oversell. The project must include a concurrency test proving no oversell occurs under simulated simultaneous requests.

Pricing

- Ticket price stored in **cents**.

3. User Journeys to Be Documented by Interns

Journey	Flow
Browse and reserve	User selects ticket type and quantity → system creates a short-lived reservation → user receives reservation_id and expires_at.
Complete purchase	User submits payment and Idempotency-Key → system confirms the order → user receives confirmation and ticket(s).
Payment failure	Payment fails during confirm → reservation remains until expiry or user cancels → user receives failure notification and retry instructions.
Reservation expiry	Reservation expires → user notified (optional) → inventory remains unchanged.
Refund (full order)	Organizer or support issues a full refund → order marked refunded → inventory incremented and audit recorded.

4. Business Acceptance Criteria

Functional

- Reservations return reservation_id and expires_at.
- Confirmed purchases are idempotent and produce a single order per idempotency key.
- Concurrency test demonstrates no oversell under simulated load.
- Expired reservations cannot be confirmed.
- Full-order refunds update inventory correctly.

Operational

- Background expiry job runs on schedule and clears stale holds.
- An audit trail exists for every inventory change and order lifecycle event (reserve, confirm, expire, refund).

Performance

- Typical checkout latency target: < 500 ms for reservation creation; < 2 s for purchase confirmation.

Security and Compliance

- Idempotency keys enforced and logged.
- Sensitive payment data is not stored; only tokens or references are recorded (simulated in this project).

5. Metrics to Track

Interns must include how each metric will be measured.

Category	Metric
Inventory	Remaining inventory per ticket type; count of active reservations.
Reliability	Idempotency collisions detected (duplicate key requests); oversell incidents (target: zero).
Operational	Reservation expiry lag (time between expires_at and actual cleanup).

6. Reporting Outputs Required

- Audit export for inventory changes with timestamps, actor, and reason.

7. Operational Notes

Interns should document the basics needed to operate the stack:

- How to view the audit log for a given reservation or order.
- How to manually re-run the expiry job if needed during testing.
- How to inspect Docker container logs for the API and database.

8. Rollout and Success Criteria (Docker-Based)

The service and its database must be fully containerized. Interns should be able to bring up the entire stack with a single command.

Containerization Requirements

- Provide a **Dockerfile** for the backend service using a multi-stage build (build stage + slim runtime image).
- Application must read all configuration (DB connection, hold window) from environment variables.
- Image must expose a health-check endpoint (e.g. /health) and define a Docker **HEALTHCHECK** instruction.

Docker Compose Stack

- A **docker-compose.yml** must define: the API service and a relational database.
- Use a named volume for database persistence so data survives container restarts.
- Services should start in dependency order using healthchecks rather than fixed sleep delays.

Demo

- Bring up the full stack with a single docker-compose up.
- Run the concurrency test against the running containers and confirm zero oversell.
- Live-demo: reserve → buy → refund a ticket, showing the audit log updating at each step.

Go / No-Go Criteria for Completion

- Concurrency test passes consistently; audit trail is complete and accurate; docker-compose up works from a clean checkout with no manual steps.

Prepared as an onboarding reference for the intern team working on the Event Management Backend project.